SESIÓN DE CLASE — Django REST Framework + API (FastAPI) + Proyecto Integrador

Sesión N° 3 — 7 Octubre de 2026 (Miércoles) — Bloque 1: Django REST Framework

Instructor	César Augusto Moreno Mena	**Centro**	CESGE
Curso	Django REST Framework + API (FastAPI) + Proyecto Integrador	**Año**	2026

N°	Fecha	Día	Semana	Bloque	Tema de la sesión	Teoría	Ejemplo	Práctica
3	7 Octubre	Miércoles	S2	Bloque 1: Django REST Framework	**Django Admin**	Registro de modelos en el panel de administración, personalización de list_display, filtros y búsqueda.	Configuración del admin para gestionar los modelos creados.	Personalización del panel admin del proyecto (list_display, search_fields, filtros).

1. Objetivo de la sesión

Al finalizar esta sesión, el aprendiz será capaz de **registrar modelos en el Django Admin**, crear un **superusuario**, y **personalizar el panel** con `list_display`, `list_filter` y `search_fields` para gestionar la información de forma eficiente.

2. ¿Qué es el Django Admin?

El **Django Admin** es un panel de administración que Django construye **automáticamente** a partir de tus modelos. Sin escribir ni una sola vista ni un solo formulario HTML, obtienes una interfaz web completa para crear, ver, editar y eliminar registros.

> **Idea clave**
> El Admin es una herramienta de **gestión interna** (para ti, tu equipo o el cliente que administra los datos), no la interfaz que verán los usuarios finales de la aplicación. Es como el "detrás de cámaras" de tu sistema.

3. Paso 1: Crear un superusuario

Para entrar al panel de administración necesitas una cuenta con permisos totales.

```
python manage.py createsuperuser
```

El sistema te pedirá:

1. **Username** (nombre de usuario, ej. `admin`).
2. **Email** (puede dejarse vacío presionando Enter).
3. **Password** (no se muestra en pantalla mientras escribes, es normal, sigue escribiendo).
4. **Password (again)**: repite la misma contraseña para confirmar.

> ⚠ **Error común**
> Si tu contraseña es muy simple (ej. "1234"), Django te preguntará "*Bypass password validation and create user anyway?*". Puedes escribir `y` para continuar en un entorno de práctica, pero en un proyecto real usa siempre contraseñas seguras.

4. Paso 2: Registrar un modelo básico

Abre `inventario/admin.py` y escribe la forma más simple de registro:

```
from django.contrib import admin
from .models import Categoria, Producto

admin.site.register(Categoria)
admin.site.register(Producto)
```

Ahora ejecuta el servidor (`python manage.py runserver`) y entra a `http://127.0.0.1:8000/admin/`. Ingresa con el usuario y contraseña que creaste. Ya deberías ver **Categorias** y **Productos** listados, con opción de agregar, editar y eliminar registros.

5. Paso 3: Personalizar el panel de administración

El registro básico funciona, pero es limitado: por ejemplo, en la lista de productos solo verás `Producto object (1)` repetido si no configuraste `__str__`, y no podrás filtrar ni buscar. Para mejorar esto, creamos una clase `ModelAdmin`.

```
from django.contrib import admin
from .models import Categoria, Producto

@admin.register(Categoria)
class CategoriaAdmin(admin.ModelAdmin):
    list_display = ('id', 'nombre')
    search_fields = ('nombre',)


@admin.register(Producto)
class ProductoAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'precio', 'stock', 'disponible', 'categoria')
    list_filter = ('disponible', 'categoria')
    search_fields = ('nombre',)
    ordering = ('nombre',)
```

¿Qué hace cada opción?

□	`list_display` : define **qué columnas** se muestran en la tabla de listado (por defecto solo se ve el `__str__`).
□	`list_filter` : agrega un panel lateral de **filtros rápidos** (ej. filtrar productos disponibles/no disponibles, o por categoría).
□	`search_fields` : agrega una **barra de búsqueda** que busca coincidencias en los campos indicados.
⇅	`ordering` : define el **orden por defecto** en el que aparecen los registros.

> ⚠ **Error común**
> `list_filter` solo funciona bien con campos de tipo booleano, fecha, opción (choices) o relaciones (ForeignKey). No lo uses con campos de texto largo como `TextField` .

6. Ejercicio práctico (para hacer en clase)

Sobre el proyecto que vienes construyendo desde el 2 de octubre, realiza lo siguiente:

1. Crea un superusuario (si aún no lo has hecho).
2. Registra tus modelos `Categoria` y tu modelo principal en `admin.py` , usando `@admin.register` (no la forma básica).
3. Configura `list_display` para tu modelo principal mostrando **al menos 4 columnas** relevantes.
4. Agrega un `list_filter` usando el campo booleano o la relación `ForeignKey` que definiste la sesión pasada.
5. Agrega `search_fields` para poder buscar por nombre.
6. Entra al panel `/admin/` , crea 2 registros nuevos directamente desde ahí, y verifica que el filtro y la búsqueda funcionen correctamente.

7. Errores comunes y cómo evitarlos

✗	**El modelo no aparece en `/admin/`** : olvidaste registrarlo en `admin.py` , o la app no está en `INSTALLED_APPS` (ver sesión del 2 de octubre).
✗	**"AlreadyRegistered"**: intentaste registrar el mismo modelo dos veces (por ejemplo, con `admin.site.register()` y también con `@admin.register`). Usa solo una de las dos formas.
✗	**No recuerdo mi contraseña de superusuario**: puedes crear otro con `python manage.py createsuperuser` usando un nombre de usuario distinto.
✗	**Los registros se ven como "Producto object (1)"**: falta el método `__str__` en el modelo (ver sesión del 5 de octubre).

8. Resumen de la sesión (checklist de aprendizaje)

☐	Sé crear un superusuario.
☐	Sé registrar un modelo en el Admin de la forma básica y con `@admin.register`.
☐	Sé usar `list_display` para mostrar columnas útiles.
☐	Sé agregar filtros con `list_filter` y búsqueda con `search_fields`.
☐	Puedo gestionar (crear, editar, eliminar) registros desde el panel de administración.

Próxima sesión (8 de octubre): sesión de repaso y práctica integradora, donde consolidaremos Introducción a Django, Modelos/ORM y Django Admin en un solo ejercicio completo.